

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-af54715d02ba9d0fb.jsonl`  
- SHA-256: `10483908faa32307e8f61047a993391584bafc4c286d4e63f7b3d1d17cb8ab5e`  
- Source modified: `2026-06-14T10:45:03+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `subagents`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T10:41:52.512Z)

Produce a discussion-ready PLAN for OWNED-MODEL PHASE-0 (the owner will discuss it with me).
PLAN ONLY ? do NOT write code or open PRs; write the plan to
`scratch/OWNED\_MODEL\_PHASE0\_PLAN.md` (scratch only). Repo cwd C:/Users/avidu/Projects/badminton-highlight-indexer.

Read: docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md (the AUTHORITATIVE roadmap ? esp. Phase-0 / M0.x milestones + the **M0.3 Gemini-silver purge** hard blocker), docs/NEXT_STEPS.md, docs/PERSONALIZATION_PLAN.md (the per-user head ties in here), CLAUDE.md (licensing guardrails: MIT/Apache/BSD-only for code+data+weights; **Qwen3-VL Apache-2.0 primary / InternVL3 MIT fallback; Gemma=AMBER; paid-LLMs NEVER a training source; exclude minors**), backend/eval/distill_local.py + classifier.py + fusion_golden.py (the existing Gen-0 lightweight-head + WASB-substrate machinery), and the corpus situation (owned n=6 golden + Roora growth).

The plan must cover: (1) the Phase-0 GOAL + greenlight/'done' criteria; (2) the **M0.3 Gemini-silver PURGE** ? the hard pre-flight blocker before ANY training run (exactly what to retarget, why); (3) the BASE choice ? Gen-0/Gen-1 = frozen WASB substrate (MIT) + an OWNED temporal head (the lightweight LogReg?ActionFormer path) and WHY the heavyweight Qwen3-VL / on-device VLM is DEFERRED; (4) DATA ? owned n=6 + Roora growth + licensing/minors guardrails + the consented-data moat; (5) how it ties to the personalization head (per-user = a thin overlay/fine-tune on the Gen-0 baseline); (6) milestones + the honest ship gate (LOVO floor, sign-test); (7) RISKS (small-N, the M0.4 ActionFormer feature-schema change that invalidates per-user LogRegs, compute/budget = largest open risk). Honest + concrete, cite the docs.

Output: write `scratch/OWNED\_MODEL\_PHASE0\_PLAN.md`; return a ~300-word exec summary + the **top 5 discussion points** for the owner.

2. assistant (2026-06-14T10:41:55.217Z)

I'll produce the Phase-0 plan. Let me first pull the repo for freshness, then read all the cited docs and code.

3. user (2026-06-14T10:41:58.731Z)

Already up to date.

---BRANCH---

master

---HEAD---

e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)

4. user (2026-06-14T10:41:58.747Z)

```
1      # Owned-model implementation plan (roadmap) ? DRAFT for owner review  
2  
3      **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner
```

```

greenlights a specific
4     milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
training source).
5     **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO +
learned prototype
6     (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy
workflow (2026-06-09).
7
8     ## Design principle (owner directive): *envision now, build later*
9     Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be
**architecturally envisioned
10    now** so we never hit a mammoth refactor. We **design the seams** (data contracts,
export boundaries, the
11    event store) up front and **implement behind them** when each milestone is greenlit.
Punting a *feature* is fine;
12    punting its *seam* is not.
13
14    ## North Star (reframed per PR #61 review)
15    - **Immediate deliverable:** a **training framework + harness** that produces a model
with **very high precision/
16    recall for extracting highlights + rallies from a clip**. *Quality is the first-order
objective.*
17    - **Eventual product:** a **single, sport-agnostic, conversational** video model ?
upload a clip, then ask
18    contextual questions ("longest rally", "make a clip of all service faults", "rallies
over 20 shots").
19    - **On-device is LATER** (a model-compression goal so power users save upload
cost/hassle) ? definitely on the
20    roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21    - **The moat = the consented dataset + the eval harness, not the weights.**
22
23    ## Hard guardrails (non-negotiable ? "never corrupted")
24    - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT
/ Apache-2.0 / BSD only.**
25    (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-
restrictive: Gemma 3 (viral),
26    **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU
cap), Commons-Clause repos.
27    - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public
grounding-CoT datasets that
28    were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision
(a real, binding cost).
29    - **Exclude minors; per-user training data needs a separate explicit opt-in; split data
by match.**
30    - ? **Base-model pretraining provenance is unauditabile for *every* open VLM (incl.
Qwen3-VL).** Our "no paid-LLM
31    training" rule is enforceable at *our fine-tuning data* layer, not the base's
pretraining ? an **explicit owner
32    risk-acceptance** is required to use any open VLM. (Governed by the base's license,
not our pipeline.)
33
34    ## The model strategy (verified 2026-06-09)
35    **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge
(runs SFT/GRPO, emits
36    weights); a *base* is the metal you forge. You pick one framework and feed it any clean
base.
37    - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio
+ timestamp-grounding +
38    GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core;
*avoid its AGPL Studio UI*)
39    as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-
time snapshot ? re-verify at impl.
40    - **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-
video context, purpose-built
41    **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm

```

the exact size's LICENSE

42 pre-release. ****Gemma-4 stays AMBER and bench-only**** ? its Apache grant is likely but its Prohibited-Use posture is

43 unconfirmed (counsel needed), ***and*** its video/timestamp capability is disputed; not the primary extractor.

44 ****InternVL3 (MIT/Apache)**** = clean fallback if a Qwen blocker appears. ? base swap is cheap ***only within**

45 timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-specific).

46 - ****START WITH THE TAL HEAD, NOT THE VLM**** (the decisive call): fine-tuned VLMs ****miss strict temporal boundaries****

47 (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries").

48 ****Boundary accuracy IS the product.**** The head (``rally_seq_proto.py``) is already de-risked on our corpus (held-out

49 AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super (\$0), is ~1M params (trivially compressible later),

50 and ****doubles as a pseudo-label teacher + the honest baseline the VLM must beat.**** So: ****head now ? VLM later****,

51 strictly sequenced. When the VLM comes, prefer ****two-stage**** (VLM coarse-proposes, head refines boundaries) over

52 wholesale replacement. ***(Open: the VLM-ceiling number is open-domain; our amateur footage may differ ? re-measure.)***

53

54 **## Conversational video-QA ? build the bookkeeping NOW, punt the model**

55 ****Punt the conversational *feature***** until extraction quality is solid (current LOVO ~0.583 ? "very high"; a chat UI

56 over a weak extractor exposes the weakness). ****Reject end-to-end-VLM-answers-each-turn**** (expensive; bad at exact

57 counting; the strong teachers are license-forbidden). ****But build the *seam* now**** (the brief requires it):

58 - ****Architecture = "extract once, query many":**** EXTRACTION (TAL head ? later VLM) emits structured timestamped

59 spans ? a durable ****per-video EVENT STORE**** ? a ****structured QUERY layer**** (text-to-SQL + ffmpeg clip-cut). The

60 cited queries ("longest rally", "all service faults", "rallies > 20 shots") are ****structured-numeric ? SQL, not a**

61 VLM job** (SQL counts exactly where VLMs hallucinate).

62 - ****Designed-now seam:**** make the ****per-video event-index schema a first-class data contract**** alongside the M0.1

63 corpus spine: per rally ``{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags,`

64 `derived_clip_ref, provenance, confidence}``. ****Reserve the event/fault-tag taxonomy (e.g. `service\_fault`) up front****

65 even if unpopulated. ? This is a real DB delta (a ``PRAGMA user_version`` migration): ``play_segments`` today has

66 ``video_id/start_time/end_time/duration/server/receiver/metadata`` only; ``highlights`` isn't a table yet ? so it's an

67 ***extension with new columns + a highlights table***, not a no-op. ****Punt**** the NL parser/agent until quality clears the bar.

68 - Paid Gemini may answer genuinely open-ended edge questions ****at inference**** over our structured index (allowed ?

69 it reasons over our data, it is not trained on its outputs).

70

71 **## Multisport (single sport-agnostic model)**

72 - ****Model/framework: UNCHANGED.**** One shared TAL head + one Qwen3-VL base, fine-tuned across sports (sport = an input

73 feature / per-sport calibration). The collector + indexer are already sport-generic. ? ****ACTION: rename the repo off**

74 "badminton" ASAP** (owner emphatic) ? consistent with this.

75 - ****What multisport changes = DATA + DETECTORS** (scale up, the accepted "good problem"): ****each new sport needs its own**

76 labeled matches (split by match) + a ****clean trajectory detector**** to feed the head's features. ****The binding blocker**

77 is per-sport detectors: ****WASB (badminton) has the unresolved **C3** checkpoint-**

```

provenance issue, and **clean MIT/
78     Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified.** Sequence by public-data
79     richness: **badminton ? tennis/TT ? pickleball/padel.**
80     - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81     and the evidence once cited for it (RacketVision) was *refuted on verification* (it's
ball-tracking, not temporal). The
82     concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83     corpus exists. Don't cite RacketVision as evidence.
84
85     ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86     - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87     - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-
process (ffmpeg, WASB-in-WSL, cloud
88     GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89     ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90     - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language.** train in
91     Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92     runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression
93     target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94     - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95     separate service. **None exists today.**
96
97     ---
98
99     ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) .
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg . TAL
ActivityNet-JSON . VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus . **DONE (n=6), running** . *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) .
*greenlight item*
115     - **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116     **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV

```

```

labels ? `output/local_rally_model.json`)
117     and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118     source-agnostic* function ? there is no store by that name; the earlier
"training.label_candidates" pointer was wrong.)
119     Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one
120     command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini =
eval/reference/fallback only.
121     - **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
122     blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean MIT/Apache detector swap.
123     ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our
temporal labels can't train the
124     shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies
per sport** (incl. any TT-on-WASB use).
125     Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
126
127     ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) . *greenlight item*
128     Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
129     rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
130     reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
131     heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
132     F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
133
134     ---
135
136     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
137     Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
138     extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
139
140     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
141     - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe
142     (PR #35) found it weak (~2.2x discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
143     parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
144     - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
145     our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
146     - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) *** a real
147     VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 6?8 GPUs ? **budget is the
148     largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
149
150     ---
151
152     ## Decisions / status
153     | # | item | status |
154     |---|---|---|
155     | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |

```

```

156 | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157 | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158 | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159 | 5 | Rename repo off "badminton" | **owner action** ? |
160 | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161
162 ## Open risks / unknowns (none block Phase-0 research)
163 Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
164 trajectory detectors not yet identified · base-model pretraining provenance unauditale
(risk-acceptance) · real VLM
165 VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
166 training · on-device 4B latency on 8 GB unmeasured.
167
168 ---
169 *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170 next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
171

```

5. assistant (2026-06-14T10:42:02.572Z)

Now let me read the other cited docs and code files.

6. user (2026-06-14T10:42:03.546Z)

```

1 # Next steps (live) ? owned-model + multisport
2
3 **Updated:** 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness
design docs merged,
4 owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior:
2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
5 ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-
first delivery ? see
6 `docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**;
**PR-2
7 upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload
MD5 byte-identity +
8 compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep
#90?#98 landed** ? ruff/mypy/
9 coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin
collapse into
10 `trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626
re-verified exact).
11 **Authoritative roadmap:**
12 `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-
blow:** memory
13 `current-state.md` / `session-handoff.md`. This file = the prioritized "what to do
next," refreshed each session.
14
15 ## Where we are (one paragraph)
16 ADR-008 ratified ("repo split driven by productionization, not isolation"): training
lives in-repo (`training/`)
17 behind a CI-enforced import wall; the featurizer is single-sourced
(`backend/features/rally_seq.py`, train==serve);
18 the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest
output/human_lovo_manifest.json --floor
19 eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0
(LOVO **0.626**, floor passed,
20 sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned
corpus is in the collector**
21 (262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet

```

```

reclassification). The **Gemini
22     battery is done** (see table) ? the moat is quantified.
23
24     ## The quality table that now drives strategy (IoU 0.5 vs human golden labels)
25     | Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
26     |---|---|---|
27     | Heuristic (velocity windowing) | 0.657 | 0.157 |
28     | **Gen-0 owned head** (LOVO, n=6) | 0.744 | 0.561 |
29     | Two-stage WASB+Gemini refine | 0.83 | ? |
30     | **Gemini wrapper** (`gemini-flash-latest`) | **0.93** | **0.66** |
31
32     **Reading:** clean footage is commoditized (serve it with Gemini for cents). **Multi-
court drops the commodity
33     wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game
pickups + dead-time merges,
34     the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-
growth-sized gap, not an
35     architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG
tapes (uploads candidate clips
36     only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry
load-bearing.
37     Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json,
gemini_wrapper_2026-06-10.json).
38
39     ## Rally-detection quality track (NEW 2026-06-13 ? design merged, owner-gated, NOT
started)
40     **This is the track a desktop/local agent is slated to pick up.** Three design docs
landed (PR #112 =
41     fusion + Roora v8; this session = experiment harness + handoff). All implementation is
**owner-gated**.
42     - **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio cues, one
fusion layer) .
43     `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) .
44     `docs/ROORA_LABELING_GUIDELINES.md` (v8) . `docs/HOW_RALLY_DETECTION_WORKS.md` (mental
model).
45     - **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a
rally" bug is
46     **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already
implements the
47     net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-
feed/held-shuttle
48     windows, but it is **only called from eval drivers** (`distill_local.py`,
`eval_partial_labels.py`,
49     `export_wasb_segments.py`, `calibrate_local.py`) and there is **no `min_crossings`
knob in
50     `IndexingConfig`** ? so production never applies it. **Wiring Gate A into the live
segmenter is the
51     single cheapest, highest-confidence quality win.**
52     - **Pick-up order (each = ONE PR off `master`, owner approval first):**
53     1. **Fusion P2 first-step ? wire Gate A into production:** add `min_crossings` to
`IndexingConfig`;
54     apply `rally_gate` in `wasb_hybrid`/`trajectory_hybrid` runtime (today it's eval-
only). Pure-logic,
55     testable in this container. *Fixes the observed bug.*
56     2. ? **DONE (2026-06-13) ? Fusion P1 ? extract the person cue:** lifted torchvision
detector + ByteTrack
57     + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py` (`PersonDetector`);
58     `yolo_hybrid` now orchestrates only + delegates to `self.person` (no behavior
change ? regression-safe,
59     proven by unchanged process_video tests). `tests/test_person_detector.py` added;
person_detector
60     type-clean. **The fusion segmenter (P2) reuses this producer over shuttle-seeded
windows.**
61     3. ? **DONE (2026-06-13) ? Experiment-harness P1 ? `backend/eval/experiment.py`:**

```

```

thin wrapper over
62         the EXISTING `calibration.py`, `metrics.py`, `classifier.py`, `training.py`,
`regression.gt_hash`, and
63         `database.query_candidate_segments` (no rebuilt scoring). Shipped `Variant` (+
pinned `CONTROL`),
64         `compare` (labeled A/B, LOVO-honest for learned variants), `compare_unlabeled`
(**SxS on new
65         unlabeled footage** ? agreed/A-only/B-only), `record_experiment` (leaderboard
JSON), and the DB
66         bridge; 19 tests incl. the no-regression invariant. Suite 421?440 green.
67         4. ? **DONE (2026-06-14) ? Fusion P2 ? the FusionSegmenter:**
`fusion_hybrid.py::FusionSegmenter`
68         (registry `fusion_hybrid`, **default-OFF**) seeds windows from the shuttle
substrate, persists
69         `net_crossings` (Gate A) + optional person features (`fusion_features.py`, pure-
tested) via two
70         identity hooks on the trajectory engine, with optional served Gate A/B
(`fusion.min_crossings` /
71         `min_players`, default 0) that gates the handoff while persisting the full
superset. Court mask =
72         optional `fusion.court_polygon`. Adversarially reviewed (31 agents, 19 confirmed;
NO-REGRESSION
73         verified); suite 441?467 green. Next: **P3** learned fusion (person features ?
harness `compare`
74         LOVO lift on the 6 golden videos; the eager-person-compute optimisation lands
here), then **P4**
75         audio cue via `backend/pipeline/audio/hit_detector.py`.
76         - **Discipline (from EXPERIMENT_HARNESS.md):** every new cue lands **flag-gated, default
OFF**; promote
77         to default **only on measured LOVO lift** through the `run_regression.py` gate (exit
0/1/3); pinned
78         control variant ? **rollback = config flip, never `git revert`**. The harness is ~80%
existing code.
79         - ? Dev container has no GPU/torch/cv2 ? Gate-A wiring, the person-cue extraction logic,
and
80         `experiment.py` are pure-logic/testable here; real-video confirmation is owner-side
(GPU/WSL).
81
82         ## Prioritized next steps
83         1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move;
every lever below feeds on it.
84         Priority order for new videos: **(a) multi-court badminton** (the target distribution
AND where the ship target
85         lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1
0.909; pickleball labels are
86         corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c)
capture variety per
87         `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool**
(consent guardrail). Per video:
88         label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ?
re-run the Gen-0 harness.
89         The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is
reachable this month.
90         2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480
(necessary), **multi-court
91         reference = wrapper 0.66** (the number that makes the owned model worth serving),
clean-footage ceiling = 0.93
92         (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70
on multi-court folds with
93         paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
94         3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for
**ActionFormer/ASFormer** (MIT, minutes on the
95         2070S ? still lightweight, NOT the heavyweight Gen-2 stack) inside the existing
harness; per-video threshold
96         calibration (the LOVO-vs-oracle gap is visible per fold in the harness output).

```



```

97     4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`);
merged-prototype LOVO across
98     badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-
on-WASB serving stays gated
99     by C3 (provenance multiplies per sport).
100    5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+
the two added questions:
101        "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot
at the warmest 2-3 academies
102        (consent at capture; demo served via the Gemini path or human-polish ? the reel
pattern exists:
103        `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner
ratified the
104        physical-visit approach; reframed product-feedback-first per owner spec):**
105        `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact
placeholders and recruits) +
106        `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student
script, demo-after-Q6
107        rule, ground rules, Phase-2 record-and-show-back stretch goal) +
`docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
108        capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md`
(G/A/R anchors + **pre-registered
109        decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first
visit).
110    6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified
working; confirm old-key
111        deletion in AI Studio); **rename repo off "badminton"** (owner action,
112        in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the
battery completed.
113
114    ## Pointers / artifacts
115    - Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` . Topology: ADR-008
(`docs/archives/decisions/DECISIONS.md`) .
116    Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` . Quality history:
117    `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
118    - Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact
bundle; weights gitignored,
119    manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).
120    - Corpus: collector DB (system-of-record, 6 matches/262 rallies) . eval manifest
`output/human_lovo_manifest.json`
121    (gitignored) . durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation
Setup\Collect\Trajectories\` .
122    labels: `...\Collect\Golden Labelled\`.
123    - Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
124    (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a
cold/prepaid project burst-throttles
125    with a MISLEADING "credits depleted" 429; ramp gradually.
126    - ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next
milestone. Committed next
127    PLATFORM slice = async job model (single-tenant).
128

```

7. user (2026-06-14T10:42:03.829Z)

```

1    # Personalization Plan ? a per-user feature on a generalization-first baseline
2
3    **Status: DESIGN ? owner-adopted (hybrid); Phase-0 engineering landing.** This is the
authoritative
4    plan for the personalization track. The detailed strategy analysis + the full pros/cons
that led here
5    live in the working artifacts `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` (the decision-
grade
6    counter-case) and `scratch/PERSONALIZATION_FEATURE_DESIGN.md` (the code-grounded
design); this doc is
7    the checked-in summary + decisions + phased build plan. Peer to

```

```

8      [OWNED_MODEL_IMPLEMENTATION_PLAN.md](OWNED_MODEL_IMPLEMENTATION_PLAN.md),
9      [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md), and
[MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md).
10
11      ## 0. Thesis & strategic rationale (why *hybrid*, not personalization-first)
12
13      **Foundation principle (owner):** "a tool that gets better the more you help it."
Contribution
14      (data / video / diagnostics / annotation) improves BOTH the user's own model AND the
shared baseline;
15      **payoff scales with contribution.**
16
17      A directional pivot ? *personalization-first* (ship a baseline that continually self-
optimizes to one
18      user's own footage) ? was researched in depth. The verdict was **HYBRID, not all-in**:
19
20      - **Adopt personalization as a flag-gated FEATURE on a generalization-first baseline.**
The harness
21      genuinely makes per-user *tuning* near-free (it is already a corpus-relative A/B
engine; a per-user
22      corpus is just a special case). This is real and worth shipping.
23      - **Do NOT bet the strategy on per-user *learning*.** Per-user A/B is the n?6 small-
sample regime
24      *worse* (sharding data when you should pool); the differentiated parts (a learned per-
user model
25      serving path, the correction?label write path, a user/tenant dimension) are largely
unbuilt; a
26      naively-overfit per-user model loses the cross-user data flywheel and has no shared
baseline to
27      regress against.
28      - **The owner's contribution-flywheel + free-tier-pooling closes the one real hole** the
counter-case
29      found: by having (free-tier) users contribute to the *shared* baseline, the
compounding cross-user
30      moat is preserved *alongside* personalization ? personalization **and** flywheel, not
either/or.
31
32      Net: a generalization-first **baseline** ("batteries"), a thin **per-user tuning** layer
gated by the
33      honest small-N statistics (with a hard floor + structural-guard protection), and a
**contribution
34      loop** that feeds both.
35
36      ## 1. Owner decisions (locked 2026-06-14)
37
38      1. **Identity** ? auth WILL land; *design for the north star (multi-user / cloud),
implement for
39      current use (single-user local)*. `user_id` nullable now (NULL = local install),
device/install
40      UUID for the alpha; real auth + a `users` table later.
41      2. **Per-user model storage** ? inline `classifier_json` in the (local) DB now, but
**behind a load
42      interface** so that post-auth it is fetched from the **cloud-hosted DB run by the
service** that
43      generates base models and orchestrates the product. The local store is one
implementation of that
44      interface.
45      3. **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate.** A
brand-new user with
46      one video is ALWAYS fully segmented by the baseline batteries ? never denied, never
"give me more
47      videos first." `min_n` only gates whether a *per-user* A/B is trusted enough to
OVERRIDE a baseline
48      cue default *for that user*; below it (or if the CI / sign test don't clear) ? keep
the baseline.

```

```

49      **Training-corpus policy:** *no more than needed, no less than all-if-few* ? fit on
the
50      `smallest_k_with_lift` videos (the held-out-USER learning curve), else ALL available.
51      4. **"Flip Gate-A to default"** = ship the validated Gate-A held-shuttle guard as the
**default
52      detection path** (today it lives in the opt-in `FusionSegmenter`, which is not the
default
53      segmenter). Changes served behavior ? **gated on a golden-set regression run + owner
OK**; still
54      open (see §5).
55      5. **Flywheel = backend property** (not in-product marketing). A **free tier** is likely
and
56      **free-tier videos ARE pooled** into the baseline (consent baked into the free terms;
57      minors-exclusion still enforced) ? the clean value-exchange that feeds the owner-
trained base model.
58      Paid tiers get stronger privacy / opt-out.
59
60      **Pending sub-decision:** structural-guard scope ? recommended **Gate-A only** (the
proven structural
61      held-shuttle guard; its value is in failure modes a user's corpus may not yet contain),
with **Gate-B**
62      (person-occupancy / `min_players`) left *subject* to per-user A/B; **`min_n = 5`**.
**Gate-A-only
63      structural, `min_n = 5` ? owner-confirmed 2026-06-14.**
64
65      ## 2. Architecture
66
67      **Two-layer decoupling (already real).** Detection (GPU, run once): `FusionSegmenter`
persists the
68      **full superset** of per-window features into `candidate_segments.stats`; only the
served handoff is
69      gated. Decision (CPU, swappable): `experiment.Variant` + `predict()` re-score persisted
features.
70      **Personalization == a per-user `Variant`** (cue_flags + `min_crossings`/`min_players` +
optional
71      LogReg + threshold). No segmenter rewrite ? a per-user `FusionConfig` overlay resolved
upstream.
72
73      **The single highest-leverage wiring:** `fusion_hybrid.py::_select_for_handoff` is the
production
74      decision seam. When a per-user model is pinned AND the promotion gate says ON, apply
75      `LogisticRegression.predict_proba` over `window_stats` (the same math
`experiment.predict()` runs
76      offline). `LogisticRegression.load` is **never called in `backend/pipeline/` today** ?
the serving
77      bridge is the #1 wall.
78
79      **Per-user data model** (migrations follow the `PRAGMA user_version` ladder; current =
v3):
80      - **v4** ? nullable `user_id` on `videos` + `candidate_segments` (NULL = local; byte-
identical to today).
81      - **v5** ? `user_models` store: `(user_id, version, baseline_version,
feature_schema_hash,
82      fusion_config, classifier_json, promotion_evidence, n_videos_at_fit, is_active)`.
Inline
83      `classifier_json` (tenant-safe); `feature_schema_hash` is the MINOR-vs-MAJOR upgrade
switch.
84      - **v6** ? `contribution_consent` ledger (default-deny; minors-exclusion enforced **at
the pooling
85      query**, not advisory).
86
87      ## 3. Constructs (the gap list + what is built)
88
89      | # | Construct | Status |
90      |---|---|---|

```

```

91 | 1 | Per-user store + serving bridge (wire `LogisticRegression.load` into
`_select_for_handoff`) | Phase 1 (the #1 wall) |
92 | 2 | Correction-loop ? per-user labels (`human_label` setter + confirm/reject API;
recall-protecting de-bias prior) | Phase 2 (`human_label` is a read-only stub today) |
93 | 3 | **Per-user promotion gate** (min-N floor + structural-guard exemption) | ? **DONE
? PR #141** (`backend/eval/per_user_gate.py`) |
94 | 4 | Contribution flywheel (opt-in data/video/diagnostics ? shared baseline; payoff ?
contribution; consent + minors-exclusion) | Phase 2 |
95 | 5 | User-as-annotator charter (rally-annotator: auto-generate a ~2-min high-
uncertainty segment from the user's own video; guide labeling ? grounding) | Phase 3 |
96 | 6a | **Held-out-USER learning curve** (the falsifiable "<K videos to superior quality"
criterion) | ? **DONE ? PR #142** (`backend/eval/per_user_eval.py`) |
97 | 6b | Per-user no-regression guard (frozen per-user CONTROL; never accept a regressing
retrain/upgrade) | ? **in flight** (`feat/per-user-no-regression`) |
98 | 7 | Day-0 "batteries" = the validated label-free Gate-A/B fusion (?F1 +0.074, 6/6,
p=0.031) shipped as the default | Phase 0, owner-gated ($5) |
99
100 ## 4. Phased PR-sized build plan + status
101
102 **Phase 0 ? safety + day-0 batteries** (no schema / API / served-behavior change; pure-
Python + tests):
103 - PR-0.1 per-user promotion gate ? ? **#141 merged**
104 - PR-0.2 held-out-USER learning curve ? ? **#142 merged**
105 - PR-0.3 per-user no-regression guard ? ? in flight
106 - PR-0.4 ship Gate-A as the day-0 battery ? **owner-gated** ($5; needs a golden-set
regression run)
107 - PR-0.5 within-user LOVO measurement (the "confirm-or-kill" experiment) ? **owner-
gated** (needs multi-video-per-user data on the GPU box)
108
109 **Phase 1 ? per-user tuning serving + store:** v4 migration +
`set_human_label`/`set_user_id`; v5
110 `user_models` + CRUD; `serving.py` overlay resolver + `load_user_classifier`; **wire the
bridge into
111 `_select_for_handoff` (owner-gated ? served path)**; `/retune` + `/model` trust-feature
endpoints.
112
113 **Phase 2 ? correction loop + flywheel:** `POST /api/candidates/{id}/label` (the first
`human_label`
114 writer); recall-protecting de-bias prior; v6 consent ledger + minors-exclusion; **opt-in
contribution
115 upload (owner-gated ? privacy surface)**; free-tier pooling.
116
117 **Phase 3 ? annotator charter + taxonomy + baseline upgrades:** active-learning task
picker; rally-
118 annotator CSV ingest ? per-user labels; **coarse hard/soft shot proxy (owner-gated)**;
**baseline-
119 upgrade governance (MINOR carry / MAJOR re-fit on `feature_schema_hash`) + optional
federated nudge
120 (owner-gated, counsel).**
121
122 ## 5. Open decisions (owner)
123
124 1. **Structural-guard scope + `min_n` value** ? lock **Gate-A-only structural, `min_n` =
5** (recommended)?
125 2. **Gate-A default flip (PR-0.4)** ? Gate-A lives in the non-default `FusionSegmenter`,
so "ship as
126 default" is a path choice (make that segmenter the default vs port the gate into the
default path).
127 Recommended sequence: **run a golden-set regression first**, bring the number, *then*
approve.
128 3. **Per-user `user_id` for the alpha** ? device/install UUID acceptable, or must auth
land first?
129 4. **Local-install flywheel default** ? fully-local zero-pooling until consent is
explicitly granted?
130

```

```

131     ## 6. Honest risks (carried from the analysis)
132
133     - **Small-N per-user eval noise** is central and *worse* per-user ? mitigated
structurally: the gate
134         refuses below `min_n` and falls back to the baseline; never auto-promotes in the
regime the tooling
135         exists to refuse in. PR-0.5 confirms-or-kills the core premise before any serving
build-out.
136     - **Cold-start** ? the day-0 Gate-A battery (zero per-user labels) is the floor for the
median user.
137     - **Privacy deferral** ? do not advertise "data never leaves your machine" until on-
device ships;
138         near-term pooling carries the full cloud (DPA / consent / minors) posture.
139     - **Compute** ? heavyweight VLM personalization (on-device Qwen3-VL fine-tune) is **out
of v1 scope**;;
140         v1 is the cheap LogReg / threshold / cue-ON-OFF layer only.
141
142     ## 7. References
143     - `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` ? the decision-grade strategy analysis
(pros/cons, harness-fit, counter-case, recommendation).
144     - `scratch/PERSONALIZATION_FEATURE_DESIGN.md` ? the code-grounded design (schema,
serving bridge, APIs, critical files).
145     - Built: `backend/eval/per_user_gate.py` (#141), `backend/eval/per_user_eval.py` (#142).
146     - Harness: [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md), `backend/eval/experiment.py`,
`backend/eval/eval_stats.py`.
147

```

8. assistant (2026-06-14T10:42:08.999Z)

Now let me read the code files: `distill\_local.py`, `classifier.py`, `fusion\_golden.py`, and check for `calibrate\_local.py` and the Gen-0 harness.

9. user (2026-06-14T10:42:09.918Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10         filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11         active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12         -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21         python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

```

```

31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]
39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
41 "net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44     PROPOSAL = (0.005, 1.0, 0.5)
45     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
49 Tuple[List[Interval], List[List[float]]]:
50     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
51 rows)."""
52     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
53     vt, mg, mwd = proposal
54     wins = TrackNetRunner.trajectory_to_action_windows(
55         points, fps=fps, frame_width=frame_width,
56         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
57     intervals = [(w["start"], w["end"]) for w in wins]
58     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
59 crossings
60     X: List[List[float]] = []
61     for w, g in zip(wins, gated):
62         dur = max(1e-6, w["end"] - w["start"])
63         win_frames = max(1.0, dur * fps)
64         X.append([
65             dur, # rally length (sec) ?
66             float(w["peak_velocity"]), # already normalized by
67             float(w["mean_velocity"]),
68             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
69             float(g.crossings), # net crossings (count) ?
70             the rally signal
71         ])
72     return intervals, X
73
74     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
75 List[Interval]:
76     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
77     vt, mg, mwd = BASELINE_LOCAL
78     wins = TrackNetRunner.trajectory_to_action_windows(
79         points, fps=fps, frame_width=frame_width,
80         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
81     return [(w["start"], w["end"]) for w in wins]
82
83     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
84         fps, fw = float(spec["fps"]), float(spec["frame_width"])
85         intervals, X = build_candidates(spec["trajectory"], fps, fw)
86         gts = load_golden_gts(spec["labels"])
87         y = label_candidates(intervals, gts, iou)
88         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
89 X,

```

```

86         "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}}
87
88
89     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
90                         threshold: float = 0.5) -> List[Interval]:
91         if not intervals:
92             return []
93         proba = model.predict_proba(X)
94         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95         return merge_overlaps(pos, gap_tol=1.0)
96
97
98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99             model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111             clf, base = [], []
112             for i, held in enumerate(videos):
113                 others = [v for j, v in enumerate(videos) if j != i]
114                 X = [row for v in others for row in v["X"]]
115                 y = [t for v in others for t in v["y"]]
116                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118                 sc = score(preds, held["gts"], iou)
119                 bsc = score(held["baseline"], held["gts"], iou)
120                 clf.append(sc)
121                 base.append(bsc)
122                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}")
123                 f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124             def mean(xs, a):
125                 return sum(getattr(s, a) for s in xs) / len(xs)
126             print(f"  MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
127                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
128                   f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
129             result["mean_learned_f1"] = mean(clf, "f1")
130             result["mean_baseline_f1"] = mean(base, "f1")
131
132             # Final model on ALL videos (for production / running on a fresh held-out video).
133             Xall = [row for v in videos for row in v["X"]]
134             yall = [t for v in videos for t in v["y"]]
135             final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136             assert final.w is not None # set by fit()
137             weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138             print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139             if model_out:
140                 final.save(model_out)
141                 print(f"final model (all videos) -> {model_out}")
142                 result["model_out"] = model_out
143             return result
144

```

```

145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

10. user (2026-06-14T10:42:10.365Z)

```

1     """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3     The offline learned segmenter (ADR-004) must run locally with no external API and
4     no heavy ML stack ? the project already depends on numpy, and sklearn is not
5     installed. This implements just enough: standardized features, balanced-class
6     gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7     model is a small, human-readable, version-controllable file.
8
9     It deliberately stays simple: the rally-window features are few and roughly
10    linearly separable (high player motion + sustained activity => real rally), so a
11    well-regularized linear model is a sensible, inspectable baseline. Swap in a
12    richer model later behind the same fit/predict_proba/save/load interface.
13    """
14
15    import json
16    from typing import List, Optional
17
18    import numpy as np
19
20
21    class LogisticRegression:
22        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23            self.lr = lr
24            self.epochs = epochs
25            self.l2 = l2
26            self.w: Optional[np.ndarray] = None
27            self.b: float = 0.0
28            self.mean: Optional[np.ndarray] = None
29            self.std: Optional[np.ndarray] = None
30            self.feature_names: Optional[List[str]] = None
31
32        @staticmethod
33        def _sigmoid(z: np.ndarray) -> np.ndarray:
34            # Clip for numerical stability (avoid overflow in exp).
35            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37        def _standardize(self, X: np.ndarray) -> np.ndarray:
38            return (X - self.mean) / self.std
39
40        def fit(self, X, y, feature_names: Optional[List[str]] = None,
41               class_weight: str = "balanced") -> "LogisticRegression":
42            """Fit with optional balanced class weighting (rally windows are imbalanced

```



```

43         ? many spurious candidates per real rally)."""
44     X = np.asarray(X, dtype=float)
45     y = np.asarray(y, dtype=float)
46     if X.ndim != 2 or X.shape[0] == 0:
47         raise ValueError("X must be a non-empty 2D array")
48     self.feature_names = feature_names
49
50     self.mean = X.mean(axis=0)
51     self.std = X.std(axis=0)
52     self.std[self.std == 0] = 1.0 # guard constant features
53     Xs = self._standardize(X)
54     n, d = Xs.shape
55
56     if class_weight == "balanced":
57         n_pos = max(1, int(y.sum()))
58         n_neg = max(1, int((1 - y).sum()))
59         w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60     else:
61         w_pos = w_neg = 1.0
62     sw = np.where(y == 1, w_pos, w_neg)
63
64     self.w = np.zeros(d)
65     self.b = 0.0
66     for _ in range(self.epochs):
67         p = self._sigmoid(Xs @ self.w + self.b)
68         err = (p - y) * sw
69         grad_w = Xs.T @ err / n + self.l2 * self.w
70         grad_b = float(err.mean())
71         self.w -= self.lr * grad_w
72         self.b -= self.lr * grad_b
73     return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106

```

```

107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115

```

11. assistant (2026-06-14T10:42:16.769Z)

Now let me read `fusion\_golden.py`, check `calibrate\_local.py`, the Gen-0 harness, and the featurizer.

12. user (2026-06-14T10:42:17.730Z)

```

1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12          python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31      from backend.utils.run_telemetry import RunTelemetry
32
33      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39      def _derive_video_path(spec: Dict[str, Any]) -> str:
40          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the

```

```

41     golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42     if spec.get("video_path"):
43         return str(spec["video_path"])
44     labels = str(spec["labels"])
45     if labels.endswith(".rallies.csv"):
46         return labels[: -len(".rallies.csv")] + ".MP4"
47     raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                      frame_skip: int = 3, iou: float = 0.5,
52                      log_every_sec: float = 15.0,
53                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
54         """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56         Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle}, {person}, {label}], gts}`.
57         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
58         when None a real `PersonDetector` is built (loads torchvision on first use).
59         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60         from the detection progress callback so a hard kill leaves a forensic runlog.
61         """
62         fps, fw = float(spec["fps"]), float(spec["frame_width"])
63         traj = str(spec["trajectory"])
64         points = TrackNetRunner.parse_trajectory_csv(traj)
65         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66         gts = load_golden_gts(str(spec["labels"]))
67         labels = label_candidates(intervals, gts, iou)
68         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70         axis, net_px, _span = rally_gate.estimate_play_axis(points)
71         video_path = _derive_video_path(spec)
72
73         if detector is None:
74             from backend.pipeline.detectors.person_detector import PersonDetector
75             detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79         n = len(intervals)
80         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81         print(f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82         t0 = time.monotonic()
83         state = {"last": t0}
84
85         def _progress(done: int, total: int) -> None:
86             if telemetry is not None:
87                 telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden] {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                       f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)

```

```

96         state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                            court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
last);
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                                  if os.path.isfile(s.get("trajectory", ""))
else 0))
138         os.makedirs(out_dir, exist_ok=True)
139         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141         # break extraction, so guard the build and fall back to no telemetry on any error.
142         telem: Optional[RunTelemetry]
143         try:
144             telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
label="fusion_golden")
145         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
146             telem = None
147         written: List[str] = []
148         for vi, spec in enumerate(specs, 1):
149             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151             # run left off instead of re-detecting). --fresh forces a redo.

```

```

152         if not fresh and os.path.isfile(out):
153             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                 f"-> {out} (use --fresh to redo)", flush=True)
155             written.append(out)
156             continue
157         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158         t0 = time.monotonic()
159         if telem is not None:
160             telem.event("video_start", spec["name"])
161         try:
162             data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
163         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
164             if telem is not None:
165                 telem.event("error", spec["name"], err=str(e))
166             raise
167         n_pos = sum(c["label"] for c in data["candidates"])
168         with open(out, "w", encoding="utf-8") as f:
169             json.dump(data, f, indent=2)
170         if telem is not None:
171             telem.event("video_done", spec["name"],
172                         windows=len(data["candidates"]), positives=n_pos)
173         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175             f"-> {out}", flush=True)
176         written.append(out)
177         return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

13. user (2026-06-14T10:42:17.951Z)

```

1     """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2     on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3     100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5     Pipeline being tuned (all local, no Gemini at runtime):
6         WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7         -> rally_gate (min_crossings) -> rally windows
8
9     Ground truth = Gemini full-context silver labels (one CSV per video). With >=2

```

```

10     videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11     the honest "how precise is the local pipeline on an unseen video" number. The
12     output is a single local config + the precision it buys, with NO Gemini dependency
13     shipped.
14
15     Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16     calibration.{select_best, leave_one_video_out, improvement_report}.
17
18     Run:
19         python -m backend.eval.calibrate_local --manifest videos.json
20     where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21     (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Callable, Dict, List, Tuple
28
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30     from backend.pipeline.detectors import rally_gate
31     from backend.eval.calibrate_wasb import load_golden_gts
32     from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34     # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35     LocalConfig = Tuple[float, float, float, int]
36     BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0) # today's local default (windowing,
no gate)
37
38
39     def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40         """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41         (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
42         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
43
44         def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
45             vt, mg, mwd, mc = cfg
46             windows = TrackNetRunner.trajectory_to_action_windows(
47                 points, fps=fps, frame_width=frame_width,
48                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
49             )
50             wins = [(w["start"], w["end"]) for w in windows]
51             if mc > 0:
52                 wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
53             return wins
54
55         return predict_fn
56
57
58     def local_grid() -> List[LocalConfig]:
59         return [(vt, mg, mwd, mc)
60                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
61                 for mg in (1.0, 2.0, 3.0)
62                 for mwd in (1.0, 2.0, 3.0)
63                 for mc in (0, 1, 2, 3)]
64
65
66     def build_videos(specs: List[Dict]) -> List[Dict]:
67         videos = []
68         for s in specs:
69             pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
float(s["frame_width"]))
70             gts = load_golden_gts(s["labels"])

```

```

71         if not gts:
72             raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
73         videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
gts})
74     return videos
75
76
77 def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
78     videos = build_videos(specs)
79     grid = local_grid()
80     print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
81           f"{len(grid)} configs, IoU>={iou}, optimise {metric}) ===")
82
83     # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
video).
84     per_video = []
85     for v in videos:
86         best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
87         imp = improvement_report(v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"],
iou)
88         b, t = imp["baseline"], imp["tuned"]
89         print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
90         print(f"  baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
F1={b['f1']:.3f}")
91         print(f"  best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
F1={t['f1']:.3f}")
92         f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)"
93         per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})
94
95     # Honest cross-video: pick config on the OTHER videos, score the held-out one.
96     lovo = leave_one_video_out(videos, grid, metric, iou)
97     # Baseline's held-out score (no tuning) for the honest before/after.
98     base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in
videos]
99     base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)
100
101     print("\n--- HONEST leave-one-video-out (the number to trust) ---")
102     if "error" in lovo:
103         print(f"  {lovo['error']} (need >=2 videos)")
104     else:
105         print(f"  baseline (untuned) mean held-out {metric}: {base_mean:.3f}")
106         print(f"  calibrated mean held-out {metric}: {lovo['mean_held_out']:.3f}")
107         f"± {lovo['std_held_out']:.3f} (gap {lovo['generalization_gap']:+.3f}")
108         for p in lovo["per_video"]:
109             print(f"    held-out {p['video']}: {p['held_out']:.3f} (cfg picked on
others: {p['config']})")
110     return {"per_video": per_video, "lovo": lovo, "baseline_held_out": base_mean,
"metric": metric}
111
112
113 def main() -> None:
114     ap = argparse.ArgumentParser(description="Calibrate the local-only pipeline against
Gemini silver labels.")
115     ap.add_argument("--manifest", required=True,
116                     help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
117     ap.add_argument("--iou", type=float, default=0.5)
118     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
119     args = ap.parse_args()
120     with open(args.manifest, encoding="utf-8") as f:
121         specs = json.load(f)
122     run(specs, iou=args.iou, metric=args.metric)
123
124
125 if __name__ == "__main__":

```

```
126         main()
127
```

14. assistant (2026-06-14T10:42:21.263Z)

Now let me look at the Gen-0 harness, the featurizer, and check the eval_baselines and how the manifest references labels (to understand the Gemini-silver vs human-golden distinction concretely).

15. user (2026-06-14T10:42:21.762Z)

```
1      """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3      python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4      --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6      Productionizes the rally_seq prototype per ADR-008:
7      - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8      honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9      the FINAL model on all videos.
10     - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11         <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12         <out>/<version>/manifest.json    (the reviewable contract: feature-schema version,
13                                         calibration, training lineage + GT hashes,
14                                         eval results, attestations, gate verdict)
15     - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16       (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17       (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18       undefined => no self-declared victory). The product side owns final admission
19       (backend/eval/regression.py); this harness never grades itself into serving.
20
21     Training may import backend.* (one-way wall); backend must never import training.*.
22     """
23     from __future__ import annotations
24
25     import argparse
26     import hashlib
27     import json
28     import os
29     import subprocess
30     import time
31     from math import comb
32     from typing import Dict, List, Optional
33
34     import numpy as np
35
36     from backend.eval.classifier import LogisticRegression
37     from backend.eval.gt_loader import load_gt_intervals
38     from backend.eval.regression import gt_hash
39     from backend.eval.rally_seq_proto import (
40         labels_for, run as lovo_run, _seg_f1,
41     )
42     from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
43     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
44
45     # Post-processing constants ? part of the model's identity, recorded in the manifest.
46     CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
47     THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
48
49
50     def _git_sha() -> str:
51         try:
```



```

52         return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
53                                text=True, timeout=10).stdout.strip() or "unknown"
54     except Exception:
55         return "unknown"
56
57
58     def _sha256(path: str) -> str:
59         h = hashlib.sha256()
60         with open(path, "rb") as f:
61             for chunk in iter(lambda: f.read(65536), b""):
62                 h.update(chunk)
63         return h.hexdigest()
64
65
66     def sign_test(learned: List[float], floor: List[float]):
67         """Paired one-sided sign test: is learned > floor more often than chance?
68
69         At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
70         wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
71         pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv] # ties drop,
standard
72         wins = sum(1 for lv, fv in pairs if lv > fv)
73         n = len(pairs)
74         p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
75         return wins, n, p
76
77
78     def train_final(specs: List[Dict]):
79         """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
80         side).
81
82         The honest generalization estimate is the LOVO result, not this fit ? recorded as
83         such."""
84         vids = []
85         for s in specs:
86             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
87             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
88             X, times = features(arr, float(s["fps"]))
89             gts = load_gt_intervals(s["labels"], strict=False)
90             vids.append({"name": s["name"], "X": X, "times": times,
91                         "y": labels_for(times, gts), "gts": gts})
92         Xall = np.vstack([v["X"] for v in vids])
93         yall = np.concatenate([v["y"] for v in vids])
94         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
95         clf.fit(Xall, yall, feature_names=FEAT_NAMES)
96         thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
97         return clf, float(thr)
98
99     def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
100         """Pre-flight ship-gate. ship=True requires EVERY block to clear ? and two blocks
101         (undefined F1@tIoU target; WASB C3 provenance) are open project-wide, so today this
102         can only ever say 'floor passed, not shippable yet'. That is by design: honesty
103         over a green light."""
104         reasons: List[str] = []
105         floor_passed = None
106         sign = None
107         if floor:
108             floor_mean = float(floor["mean_f1"])
109             floor_passed = eval_res["mean_f1"] > floor_mean
110             if not floor_passed:
111                 reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
112 {floor_mean:.3f}")
113         per = floor.get("per_video", {})
114         paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if

```

```

r["video"] in per]
113         if paired:
114             wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
115             sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
116             if p >= 0.05:
117                 reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
118                               f"? grow the corpus before claiming superiority")
119         else:
120             reasons.append("no floor baseline provided ? nothing to beat")
121
122         reasons.append("ship-gate F1@tIoU target UNDEFINED (OWNED_MODEL_IMPLEMENTATION_PLAN
M0.4) ? "
123                       "beating the floor is necessary, not sufficient")
124         reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
125         return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
126
127
128     def run(manifest_path: str, out_dir: str, version: str,
129            floor_path: Optional[str] = None) -> dict:
130         with open(manifest_path, encoding="utf-8") as f:
131             specs = json.load(f)
132             floor = None
133             if floor_path:
134                 with open(floor_path, encoding="utf-8") as f:
135                     floor = json.load(f)
136
137             # 1) Honest LOVO eval (the number that matters).
138             eval_res = lovo_run(specs)
139
140             # 2) Final model on all videos.
141             clf, thr = train_final(specs)
142
143             # 3) Gate (pre-flight).
144             verdict = gate_verdict(eval_res, floor)
145
146             # 4) Artifact bundle.
147             bundle_dir = os.path.join(out_dir, version)
148             os.makedirs(bundle_dir, exist_ok=True)
149             weights_path = os.path.join(bundle_dir, "weights.json")
150             payload = clf.to_dict()
151             payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
152             payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
153             with open(weights_path, "w", encoding="utf-8") as f:
154                 json.dump(payload, f, indent=2)
155
156             manifest = {
157                 "artifact": {
158                     "name": "rally_tal_gen0", "version": version,
159                     "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
160                     "git_sha": _git_sha(),
161                     "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
162                 },
163                 "feature_schema": FEATURE_SCHEMA,
164                 "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
165                                "note": "threshold tuned train-side on the full corpus; "
166                                "honest generalization = the LOVO eval below"},
167                 "training_lineage": {
168                     "corpus_manifest": manifest_path,
169                     "split_protocol": "leave-one-video-out by match",
170                     "label_source": "human (rally-annotator golden CSVs)",
171                     "videos": [{"name": s["name"], "labels": s["labels"],
172                                "gt_hash": gt_hash(load_gt_intervals(s["labels"]),

```

```

strict=False)),
173             "fps": s["fps"]} for s in specs],
174         },
175         "eval": {
176             "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
177             "per_video": eval_res["per_video"],
178             "mean_f1": eval_res["mean_f1"],
179             "floor": floor,
180         },
181         "attestation": {
182             "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
183             "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
184             "no_paid_llm_training_labels": True,
185             "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
186         },
187         "gate": verdict,
188     }
189     manifest_out = os.path.join(bundle_dir, "manifest.json")
190     with open(manifest_out, "w", encoding="utf-8") as f:
191         json.dump(manifest, f, indent=2)
192
193     print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
194     print(f"weights : {weights_path}")
195     print(f"manifest: {manifest_out}")
196     print(f"GATE    : ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
197           f"sign_test={verdict['sign_test']}")
198     for b in verdict["blockers"]:
199         print(f"    - {b}")
200     return manifest
201
202
203     def main() -> None:
204         ap = argparse.ArgumentParser(description=__doc__)
205         ap.add_argument("--manifest", required=True,
206                         help="corpus manifest JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
207         ap.add_argument("--floor", default=None,
208                         help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
209         ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
210         ap.add_argument("--version", required=True, help="semver for the artifact bundle")
211         args = ap.parse_args()
212         run(args.manifest, args.out, args.version, floor_path=args.floor)
213
214
215     if __name__ == "__main__":
216         main()
217

```

16. user (2026-06-14T10:42:22.298Z)

```

1     """Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature
schema.
2
3     THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
4     harness (training/) and the serving path import THIS module, so train == serve by
5     construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
6     59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.
7
8     Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
9     MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
10    trained with and the serving loader refuses a mismatch.
11    """

```

```

12     from __future__ import annotations
13
14     from typing import Dict
15
16     import numpy as np
17     from scipy.ndimage import uniform_filter1d, maximum_filter1d
18
19     # Bump on ANY semantic change to build_frame_arrays/features (names, normalization,
20     # windowing, occlusion rule). Artifacts pin this; the loader hard-fails on mismatch.
21     FEATURE_SCHEMA_VERSION = 1
22
23     FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
24
25     # Default windowing ? part of the schema (manifests record the values actually used).
26     DEFAULT_WIN_SEC = 0.75
27     DEFAULT_STRIDE_SEC = 0.1
28
29     FEATURE_SCHEMA = {
30         "id": "rally_seq",
31         "version": FEATURE_SCHEMA_VERSION,
32         "feat_names": FEAT_NAMES,
33         "win_sec": DEFAULT_WIN_SEC,
34         "stride_sec": DEFAULT_STRIDE_SEC,
35         # Explicit because its absence was a real bug: speed is fraction-of-frame-width
36         # PER SECOND (* fps), matching the heuristic windowing brain.
37         "fps_semantics": "per-second",
38         "occlusion_rule": "speed/crossings not computed across gaps > max(2,
round(0.25*fps)) frames",
39         "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
frame_width}",
40     }
41
42
43     def _spread(a: np.ndarray) -> float:
44         return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0
45
46
47     def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
48         """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.
49
50         Speed is normalized to fraction-of-frame-width PER SECOND (`* fps`), matching the
51         heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
52         speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
53         on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
54         occlusion gap (mirrors the heuristic resetting on invisibility)."""
55         pts = sorted(points, key=lambda p: p.frame)
56         N = (pts[-1].frame + 1) if pts else 1
57         vis = np.zeros(N)
58         x = np.zeros(N)
59         y = np.zeros(N)
60         for p in pts:
61             if 0 <= p.frame < N and p.visible and p.x >= 0:
62                 vis[p.frame] = 1.0
63                 x[p.frame] = float(p.x)
64                 y[p.frame] = float(p.y)
65
66         mask = vis > 0
67         play_is_x = _spread(x[mask]) >= _spread(y[mask])
68         coord = x if play_is_x else y
69         net = float(np.median(coord[mask])) if mask.any() else 0.0
70         deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)
71

```

```

72     speed = np.zeros(N)
73     spd_mask = np.zeros(N)
74     cross = np.zeros(N)
75     max_gap = max(2, round(0.25 * fps)) # frames; a longer gap = occlusion break, not
motion
76     prev = None
77     for p in pts:
78         f = p.frame
79         if not (0 <= f < N and vis[f]):
80             continue
81         if prev is not None:
82             df = f - prev
83             if 0 < df <= max_gap:
84                 speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
85                 spd_mask[f] = 1.0
86                 c, pc = coord[f] - net, coord[prev] - net
87                 if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
88                     cross[f] = 1.0
89             prev = f
90     return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
91
92
93     def features(arr: Dict, fps: float, win_sec: float = DEFAULT_WIN_SEC,
94                 stride_sec: float = DEFAULT_STRIDE_SEC):
95         """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
96         N = arr["N"]
97         fw = arr["fw"]
98         size = max(3, int(win_sec * fps) * 2 + 1)
99         vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
100
101         n = uniform_filter1d(vis, size, mode="constant") # mean
visibility (== vis_rate)
102         sm = uniform_filter1d(spd_mask, size, mode="constant")
103         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
104         spd_max = maximum_filter1d(speed, size, mode="constant")
105         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps # crossings
/ second
106         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
107         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
108         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
109         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
110         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
111         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
112
113         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
114         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
115         return np.nan_to_num(X), idx / fps
116

```

17. assistant (2026-06-14T10:42:29.153Z)

I now have a thorough picture. Let me quickly verify a couple of supporting facts: the `eval\_baselines/` contents (the floor numbers), and confirm where Gemini-silver labels feed (grep for the manifest/CSV usage), so the M0.3 purge section is precise.

18. user (2026-06-14T10:42:31.245Z)

```

total 28
drwxr-xr-x 1 avidu 197609  0 Jun 14 15:54 .
drwxr-xr-x 1 avidu 197609  0 Jun 14 15:54 ..

```

```
-rw-r--r-- 1 avidu 197609 4155 Jun 14 15:54 experiment_leaderboard.json
-rw-r--r-- 1 avidu 197609 1977 Jun 10 17:43 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609 689 Jun 10 15:57 heuristic_lovo_n6.json
===HEURISTIC FLOOR===
{
  "name": "heuristic_velocity_windowing_LOVO",
  "description": "The honest cross-video floor the Gen-0 learned model must beat: calibrated
velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
  "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local --manifest
output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
  "mean_f1": 0.480,
  "std": 0.242,
  "per_video": {
    "adarsh_avi_singles": 0.751,
    "kushagra_singles": 0.157,
    "largetest_doubles": 0.737,
    "gbaaddy": 0.309,
    "testlarge_short": 0.267,
    "mahadevpura_singles": 0.657
  }
}
```

19. user (2026-06-14T10:42:32.065Z)

Found 24 files
backend\eval\ablation.py
backend\eval\fusion_golden.py
docs\CODE_MAP.md
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\EXPERIMENT_HARNESS.md
backend\eval\experiment.py
tests\test_fusion_compare.py
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
backend\pipeline\detectors\fusion_features.py
docs\DEFERRED_HARDENING_PLAN.md
backend\eval\calibrate_local.py
backend\eval\distill_local.py
docs\archives\decisions\DECISIONS.md
eval_baselines\heuristic_lovo_n6.json
artifacts\rally_tal_gen0\0.1.0\manifest.json
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
tests\test_calibrate_local.py
tests\test_distill_local.py

20. assistant (2026-06-14T10:42:38.262Z)

Note: `heuristic\_lovo\_n6.json` reproduces via `calibrate\_local` (which itself uses Gemini-silver labels in its docstring, but here it's pointed at human golden labels via the manifest). The floor mean is 0.480. The NEXT_STEPS table cites 0.626 for the Gen-0 head LOVO. Let me verify the existing Gen-0 artifact manifest's recorded numbers and check how `distill\_local`/`calibrate\_local` consume Gemini vs human labels concretely (the docstrings say "Gemini silver CSV" but the harness uses human golden CSVs).

21. user (2026-06-14T10:42:39.368Z)

```
1      {
2      "artifact": {
```

```

3      "name": "rally_tal_gen0",
4      "version": "0.1.0",
5      "created_at": "2026-06-10T15:54:44+0530",
6      "git_sha": "bfb54320fffe1b8398fbc7f62d7290e9cbfc765c",
7      "weights_file": "weights.json",
8      "weights_sha256": "75ae2d38527a79ac3a979f1040bb1f4ab0761946a6bbc597d54af32739ed6090"
9  },
10     "feature_schema": {
11         "id": "rally_seq",
12         "version": 1,
13         "feat_names": [
14             "vis_rate",
15             "spd_mean",
16             "spd_max",
17             "cross_rate",
18             "x_std",
19             "y_std"
20         ],
21         "win_sec": 0.75,
22         "stride_sec": 0.1,
23         "fps_semantics": "per-second",
24         "occlusion_rule": "speed/crossings not computed across gaps > max(2,
round(0.25*fps)) frames",
25         "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
frame_width}"
26     },
27     "calibration": {
28         "threshold": 0.5,
29         "smooth": 7,
30         "merge_gap": 1.0,
31         "min_dur": 1.0,
32         "note": "threshold tuned train-side on the full corpus; honest generalization = the
LOVO eval below"
33     },
34     "training_lineage": {
35         "corpus_manifest": "output/human_lovo_manifest.json",
36         "split_protocol": "leave-one-video-out by match",
37         "label_source": "human (rally-annotator golden CSVs)",
38         "videos": [
39             {
40                 "name": "adarsh_avi_singles",
41                 "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/Adarsh and Avi.rallies.csv",
42                 "gt_hash": "5d76c5677e5c",
43                 "fps": 59.94
44             },
45             {
46                 "name": "kushagra_singles",
47                 "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/KushagraYashAviFirstVideo.rallies.csv",
48                 "gt_hash": "0ff71939409d",
49                 "fps": 59.94
50             },
51             {
52                 "name": "largetest_doubles",
53                 "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/LargeTestVideo.rallies.csv",
54                 "gt_hash": "d58455c83d83",
55                 "fps": 29.97
56             },
57             {
58                 "name": "gbaaddy",
59                 "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv",
60                 "gt_hash": "b8d2a7d9e4a9",

```

```

61         "fps": 59.94
62     },
63     {
64         "name": "testlarge_short",
65         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv",
66         "gt_hash": "862b39766027",
67         "fps": 29.97
68     },
69     {
70         "name": "mahadevpura_singles",
71         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv",
72         "gt_hash": "c261e9efc7fa",
73         "fps": 29.97
74     }
75 ]
76 },
77 "eval": {
78     "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
79     "per_video": [
80         {
81             "video": "adarsh_avi_singles",
82             "thr": 0.44999999999999996,
83             "auc": 0.8740276491786253,
84             "f1": 0.6837606837606838,
85             "precision": 0.5797101449275363,
86             "recall": 0.8333333333333334,
87             "n_pred": 138,
88             "n_gt": 96,
89             "oracle_thr": 0.3,
90             "oracle_f1": 0.7160493827160493
91         },
92         {
93             "video": "kushagra_singles",
94             "thr": 0.44999999999999996,
95             "auc": 0.851216875529341,
96             "f1": 0.5060240963855422,
97             "precision": 0.4117647058823529,
98             "recall": 0.65625,
99             "n_pred": 51,
100            "n_gt": 32,
101            "oracle_thr": 0.49999999999999994,
102            "oracle_f1": 0.6
103        },
104        {
105            "video": "largetest_doubles",
106            "thr": 0.49999999999999994,
107            "auc": 0.915870690555361,
108            "f1": 0.7394957983193275,
109            "precision": 0.6285714285714286,
110            "recall": 0.8979591836734694,
111            "n_pred": 70,
112            "n_gt": 49,
113            "oracle_thr": 0.44999999999999996,
114            "oracle_f1": 0.773109243697479
115        },
116        {
117            "video": "gbaaddy",
118            "thr": 0.44999999999999996,
119            "auc": 0.8283100546729266,
120            "f1": 0.5303030303030303,
121            "precision": 0.4166666666666667,
122            "recall": 0.7291666666666666,
123            "n_pred": 84,

```



```

124         "n_gt": 48,
125         "oracle_thr": 0.39999999999999997,
126         "oracle_f1": 0.56
127     },
128     {
129         "video": "testlarge_short",
130         "thr": 0.49999999999999994,
131         "auc": 0.7440137189469781,
132         "f1": 0.5555555555555556,
133         "precision": 0.4166666666666667,
134         "recall": 0.8333333333333334,
135         "n_pred": 12,
136         "n_gt": 6,
137         "oracle_thr": 0.44999999999999996,
138         "oracle_f1": 0.5882352941176471
139     },
140     {
141         "video": "mahadevpura_singles",
142         "thr": 0.49999999999999994,
143         "auc": 0.8964183336830783,
144         "f1": 0.7435897435897436,
145         "precision": 0.6170212765957447,
146         "recall": 0.9354838709677419,
147         "n_pred": 47,
148         "n_gt": 31,
149         "oracle_thr": 0.44999999999999996,
150         "oracle_f1": 0.7466666666666666
151     }
152 ],
153     "mean_f1": 0.6264548179856472,
154     "floor": {
155         "name": "heuristic_velocity_windowing_LOVO",
156         "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
157         "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
158         "mean_f1": 0.48,
159         "std": 0.242,
160         "per_video": {
161             "adarsh_avi_singles": 0.751,
162             "kushagra_singles": 0.157,
163             "largetest_doubles": 0.737,
164             "gbaaddy": 0.309,
165             "testlarge_short": 0.267,
166             "mahadevpura_singles": 0.657
167         }
168     },
169 },
170     "attestation": {
171         "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
172         "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see COMMERCIALIZATION
C3)",
173         "no_paid_llm_training_labels": true,
174         "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult sessions)"
175     },
176     "gate": {
177         "ship": false,
178         "floor_passed": true,
179         "sign_test": {
180             "wins": 5,
181             "n": 6,
182             "p_one_sided": 0.1094
183         },
184         "blockers": [

```

```

185         "paired sign test not significant (5/6 wins, p=0.109) \u2014 grow the corpus
before claiming superiority",
186         "ship-gate F1@tIoU target UNDEFINED (OWNED_MODEL_IMPLEMENTATION_PLAN M0.4) \u2014
beating the floor is necessary, not sufficient",
187         "WASB checkpoint provenance C3 UNRESOLVED \u2014 commercial ship blocked
regardless of F1"
188     ]
189 }
190 }

```

22. user (2026-06-14T10:42:40.427Z)

```

33: def load_golden_gts(csv_path: str) -> List[Interval]:
34-     """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35-
36-     Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37-     start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38-     driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39-     """
40-     from backend.eval.gt_loader import load_gt_intervals
41-     return load_gt_intervals(csv_path, strict=False)
42-
43-
44- def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45-     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46-     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
47-
48-     def predict_fn(cfg: Config) -> List[Interval]:
49-         --
68-         gts = load_golden_gts(labels_csv)
69-         if not gts:
70-             raise SystemExit(f"no usable golden rallies in {labels_csv}")
71-         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
72-         grid = default_grid()
73-
74-         base = evaluate(predict_fn(BASELINE), gts, iou)
75-         best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1", iou_threshold=iou)
76-         fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC (fit on
full GT)
77-         cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-out
recall
78-
79-         print("=== WASB rally-segmentation calibration ===")
80-         print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
81-         print(f"baseline cfg {BASELINE}: precision={base.precision:.3f} recall={base.recall:.3f}
"
82-               f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83-         print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")

```

23. assistant (2026-06-14T10:42:47.645Z)

I now have everything I need. Key clarifications confirmed:

- `distill\_local.py` / `calibrate\_local.py` are **label-source-agnostic** at the code level (they call `load\_golden\_gts` on whatever CSV the manifest points to), but their **docstrings** hardcode the Gemini-silver framing** and their default output paths (`output/local\_rally\_model.json`) are the "Gemini-trained student" lineage. The M0.3 purge is about retargeting framing/defaults/lineage so no run can silently train on Gemini CSVs ? not deleting working code.
- The Gen-0 harness (`training/gen0/harness.py`) already uses **human golden CSVs** and asserts `no\_paid\_llm\_training\_labels: true`. So the harness is already on the clean path; the legacy `distill\_local`/`calibrate\_local` drivers are the residual Gemini-silver risk.
- Floor = 0.480 (heuristic LOVO), Gen-0 head LOVO = 0.626, multi-court wrapper ship target = 0.66, sign test 5/6 p=0.109 (not yet significant).

Now I'll write the plan.

24. assistant (2026-06-14T10:44:43.043Z)

Owned-Model Phase-0 ? discussion-ready PLAN (scratch)

Status: DISCUSSION DRAFT for the owner. PLAN ONLY ? no code, no PRs. Nothing here starts until the owner greenlights a specific milestone (the standing Phase-0 discipline in `docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md`).

Authority: This plan *summarizes and sequences* the authoritative roadmap `docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md` (M0.1?M0.4) and the live `docs/NEXT\_STEPS.md`; it defers to them on any disagreement. It also ties into `docs/PERSONALIZATION\_PLAN.md` (the per-user head). It does **not** introduce new product/strategy/licensing decisions ? those stay owner-gated.

Why this exists now: the foundation is mostly *built but ungated*. The Gen-0 head exists and produces a versioned artifact (`artifacts/rally\_tal\_gen0/0.1.0/`, LOVO **0.626** vs floor **0.480**, ship=False by design). Phase-0 is therefore not "build the model" ? it's "clear the pre-flight blockers and define the gate so the already-built head can be *trusted* and grown."

1. Phase-0 GOAL + greenlight / "done" criteria

Goal (one line): stand up a **trustworthy**, license-clean, \$0/local Gen-0 rally/highlight extraction baseline ? a one-command `train` ? honest-LOVO ? ship-gate ? versioned artifact` harness over a frozen MIT trajectory substrate + an **owned** lightweight temporal head ? with the compliance blockers cleared and the ship-gate target defined, so corpus growth (not method churn) becomes the only lever between here and a shippable model.

Phase-0 is explicitly **not** the conversational VLM, not on-device, not multisport. Those are **envisioned-now, built-later** (roadmap "design the seams, implement behind them").

"Done" / greenlight criteria (all four, each an owner-gated milestone):

ID	Milestone	"Done" means	Today
M0.1	Corpus spine + event-index contract	Canonical per-rally JSONL row + queryable per-video event schema; round-trip test; split-by-match enforced; zero Gemini-sourced rows in any training view; fault/event taxonomy reserved . Authored in `sports-data-collector` (system-of-record). Corpus in collector (n=6, 262 rallies); event-index schema/contract not yet formalized.	
M0.2	Grow the corpus	n=6 distinct matches labeled (3 singles, 1 doubles, 2 multi-court); per-match + LOVO recorded. DONE (n=6), running ? owner growing toward margin.	
M0.3	Compliance cleanup	(hard blocker ? before any training run) (i) Gemini-silver training path purged/retargeted ; (ii) WASB C3 provenance carried as a tracked ship -gating blocker. (i) residual risk ? see S2; (ii) owner: yes, tracked.	
M0.4	Gen-0 TAL harness + ship-gate target	One-command harness exists and the explicit F1@tIoU "very high P/R" target is ratified (a number, not just "beat 0.480"). Harness built (`training/gen0/harness.py`); ship-gate target still UNDEFINED (the open decision).	

Phase-0 is "done" when: M0.1 + M0.3 land, the M0.4 ship-gate target is ratified, and the harness re-runs green on the grown corpus. That unlocks **Phase 1** (the Gen-0 ship **decision**): run the A/B; if Gen-0 clears the now-defined gate with per-video calibration **and** the paired sign test is significant, it becomes the rally/highlight extractor (local, MIT). If not, the gap is "more data," not "more architecture."

2. M0.3 ? the Gemini-silver PURGE (the hard pre-flight blocker)

****Why it blocks everything:**** committed guardrail (`CLAUDE.md`, roadmap "Hard guardrails") ?
****paid LLMs are inference/serving/fallback ONLY, never a training data source**** (terms + copyright). Any training run that ingests Gemini labels corrupts the model's provenance irreversibly. So this is a **pre-flight** gate: it must clear ****before any training run****, not after.

****What's actually at risk (precise ? I read the code):**** nothing Gemini-trained is in git (artifacts live under gitignored `output/`), and the **current** Gen-0 harness is already clean ?
`training/gen0/harness.py` trains on ****human golden CSVs**** and asserts
`no\_paid\_llm\_training\_labels: true` in its manifest. The residual risk is the ****legacy distill drivers****, which "regenerate one command away":

- ****`backend/eval/distill\_local.py`**** ? its **entire framing** is "distill ****Gemini (teacher)**** into a LEARNED local classifier (student)." It trains a `LogisticRegression` on candidate labels and writes the default output ****`output/local\_rally\_model.json`**** ? the "Gemini-trained student" lineage. ****NB:**** the code is **label-source-agnostic** ? it calls
`calibrate\_wasb.load\_golden\_gts(spec["labels"])` on whatever CSV the manifest names, and
`training.label\_candidates` is a ****pure, source-agnostic**** function (the roadmap's earlier "training.label_candidates is the violation" pointer was **wrong** ? confirmed in M0.3 text and in the code). So the file isn't **intrinsically** dirty; its ****docstring, defaults, and output filename advertise and default toward the Gemini-silver path.****
- ****`backend/eval/calibrate\_local.py`**** ? same shape: docstring "Distill Gemini (teacher) into the LOCAL-ONLY pipeline... Ground truth = Gemini full-context silver labels." It distills **thresholds** (not weights) but is the second "trained-from-Gemini" entry point. (It is also what **reproduces the heuristic floor** `eval\_baselines/heuristic\_lovo\_n6.json` ? but pointed at ****human**** labels via the manifest, which is legitimate.)

****Exactly what to retarget (the M0.3 work ? design, owner to approve):****

1. ****Retarget the label source to human + open-teacher only.**** Make "training view = human (rally-annotator golden) ? own-model/WASB pseudo-labels" the **only** sanctioned input; Gemini = eval/reference/fallback CSVs only, never a `--model-out` lineage.
2. ****Kill the Gemini-trained default artifact path.**** Rename/relink so a `distill\_local` run can **never silently** emit `output/local\_rally\_model.json` as a shippable student off Gemini CSVs. Options to discuss: (a) gate the drivers behind an explicit `--label-source {human,open}` with no Gemini default; (b) demote `distill\_local`/`calibrate\_local` to ****eval-only**** reference tools (they already have honest-LOVO value as **baselines**) and route all **training** through `training/gen0/harness.py`, which is already clean; (c) add a ****provenance attestation**** to every emitted artifact (the harness already writes
`no\_paid\_llm\_training\_labels`) and a CI/round-trip check that the corpus "training view" has
`source ? {human, pseudo, vendor}` and ****rejects `source=gemini` rows**** (this is exactly the M0.1 acceptance test ? "no Gemini-sourced rows in any training view"). ****Recommendation: (b) + (c)**** ? one clean training door, a hard guard on it, legacy drivers kept as eval references with corrected docstrings.
3. ****Correct the docstrings.**** The "distill Gemini ? student" narrative in both files is now a **documented guardrail violation in prose**; rewrite to "honest LOVO baseline over human/open-teacher labels; Gemini CSVs accepted as a **reference** GT only."

****Also tracked under M0.3 (gates SHIP, not Gen-0 research):**** ****WASB C3**** ? the academic-data checkpoint provenance is a **commercial-ship** blocker that a clean head does NOT launder (`gate\_verdict` already hard-codes this as a blocker). Resolve via own-trained weights or a clean MIT/Apache detector swap. ? per ADR-002 our **temporal** labels can't train a **shuttle coordinate** detector ? own-weights is an ****un-scoped new workstream that multiplies per sport****. Carry it; do ****not**** let it block Gen-0 **research** (it blocks the eventual **ship**).

3. BASE choice ? frozen WASB substrate (MIT) + an OWNED temporal head; VLM DEFERRED

****The decision (roadmap "START WITH THE TAL HEAD, NOT THE VLM):**** Gen-0/Gen-1 =
****frozen WASB shuttle-trajectory substrate (MIT, commercial-clean) + an OWNED temporal head****
on top. The head is the only thing we train. Concretely the stack is already wired:

- ****Substrate (frozen, not trained):**** WASB ? per-frame shuttle trajectory CSV
(`TrackNetRunner.parse\_trajectory\_csv`). MIT. (Provenance caveat = C3, §2.)

- **Featurizer (single-sourced, train==serve):** `backend/features/rally_seq.py`
(`FEATURE_SCHEMA` v1, 6 fps/resolution-invariant features: `vis_rate`, `spd_mean`, `spd_max`, `cross_rate`, `x_std`, `y_std`). Pure numpy/scipy ? importable without torch/cv2. **Any semantic change bumps `FEATURE_SCHEMA_VERSION`**; artifacts pin it; the loader hard-fails on mismatch. (This versioning is load-bearing for \$5 / \$7 ? it's the per-user invalidation switch.)
- **Head (owned, trained):** today a dependency-free `LogisticRegression`
(`backend/eval/classifier.py`, ~6 weights, JSON-serializable) ? the M0.4 next increment swaps in **ActionFormer** (MIT, 1:1 rally=instance) / **ASFormer** (MIT) *inside the same harness*. Trains in minutes at ~4.5 GB on the 2070 Super, \$0, ~1M params.

Why the heavyweight Qwen3-VL / on-device VLM is DEFERRED (the decisive call, roadmap-cited):

1. **Boundary accuracy IS the product, and VLMs miss strict temporal boundaries.** The best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks ("coarse regions, not accurate boundaries"). A TAL head is purpose-built for exact cuts.
2. **It's already de-risked on our corpus:** held-out AUC 0.83±0.92; LOVO **0.626** vs floor **0.480** ? beating the floor with only n=6 (`artifacts/rally_tal_gen0/0.1.0/manifest.json`).
3. **\$0, local, no cloud/DPA** ? Phase-0 stays on the 2070 Super. The VLM needs rented A100/L40S (ms-swift multimodal-GRPO examples use 678 GPUs) ? **budget** is the largest open risk and is currently unquantified (\$7), so we do not stand it up in Phase-0.
4. **The head doubles as the honest baseline the VLM must beat** *and* a pseudo-label teacher. When the VLM comes (Gen-2), prefer **two-stage** (VLM coarse-proposes ? head refines boundaries) over wholesale replacement.

Envisioned-now seams (so the deferral isn't a future refactor): the event-index data contract (M0.1), the `FEATURE_SCHEMA` version pin, the artifact-bundle boundary (serving may only consume a versioned bundle), and the "extract once ? event store ? structured query" architecture for conversational-QA. We design these now and implement behind them.

4. DATA ? owned n=6 + Roora growth + guardrails + the consented-data moat

Corpus today (M0.2 DONE): 6 distinct matches / 262 rallies in the collector (system-of-record): 3 singles (incl. Mahadevpura), 1 doubles, 2 multi-court; `Proprietary`, commercial-export = owned-only (after the ShuttleSet reclassification). Per-match + LOVO recorded. **Bottleneck** = data + calibration, not method.

Growth (the single highest-leverage move ? `NEXT_STEPS.md` #1): priority order ?

- (a) multi-court badminton (the target distribution *and* where the ship number lives),
- (b) ?3 matches per new sport, table tennis first (WASB transfers at F1 0.909; pickleball labels are corpus-gold but **not trainable until a clean MIT/Apache ball detector exists**),
- (c) capture variety per `VIDEO_RECORDING_GUIDELINES.md`. **Roora** (Bangalore vendor) is the external growth engine; ingest via `curate import-local --normalize --license Proprietary --fps <true fps>` ? re-run the Gen-0 harness.

The n matters statistically (not vanity): the paired sign test is the gate's binding stat, not the mean. At n=6 / 5 wins, p=0.109 (not significant). At **n=10 / 9 wins**, p=0.011 ? **significance is reachable this month** by growing the corpus alone.

Licensing / minors / consent guardrails (non-negotiable, `CLAUDE.md` + roadmap):

- **MIT / Apache-2.0 / BSD only** for code + data + weights. Gen-0 stack is clean (numpy/scipy BSD + WASB MIT + repo-internal). Reject Gemma 3 (viral), Gemma 3n, Llama 4 (700M-MAU cap), Commons-Clause repos. **Gemma-4 = AMBER / bench-only**.
- **Never train on paid-LLM outputs** (\$2) ? **also** forbidden: public Gemini-generated CoT datasets (e.g. TVG-R1's 56K); we originate our own CoT supervision when the VLM comes (a real cost).
- **Exclude minors** from the training pool (GDPR Art. 9); **per-user training data** needs a separate explicit opt-in; **split data BY MATCH** (never leak a match across train/eval). Minors-exclusion is enforced **at the pooling query**, not advisory (`PERSONALIZATION_PLAN` v6 consent ledger).

The moat (roadmap North Star): **the moat = the consented dataset + the eval harness, NOT the weights.** The Gemini battery quantified this: clean footage is commoditized (Gemini serves it at 0.93 for cents); **multi-court drops the wrapper to 0.66** ? that is the owned model's ship

target** and the spot the moat actually defends. The `PERSONALIZATION\_PLAN` flywheel (free-tier videos *pooled* into the shared baseline, consent baked into free terms) is what compounds the corpus moat over time.

5. Tie-in to the personalization head (per-user = thin overlay on Gen-0)

Per `docs/PERSONALIZATION\_PLAN.md` (owner-adopted **hybrid**: generalization-first baseline + flag-gated per-user tuning + a contribution flywheel). The clean architectural fit:

- **Personalization** == a per-user `experiment.Variant` over the *same* persisted features ? no segmenter rewrite. A per-user model is (cue_flags + `min\_crossings`/`min\_players` + an optional per-user `LogReg` + threshold) resolved as a thin `FusionConfig` overlay upstream of the **Gen-0 baseline**. The Gen-0 head IS the day-0 "batteries" every user gets before contributing anything.
- **Decoupling** is already real: detection (GPU, once) persists the **full feature superset** into `candidate\_segments.stats`; decision (CPU, swappable) re-scores. So a per-user head is a cheap re-score, never a re-detect.
- **min_n** (owner-locked = 5) is a PROMOTION-confidence floor, NOT a service gate. A new user with one video is *always* fully segmented by the Gen-0 baseline; `min\_n` only gates whether a *per-user* A/B is trusted enough to override a baseline default for that user.
- **Phase-0** personalization work is already landing and is safety-only (no schema / served / API change): per-user promotion gate (`backend/eval/per\_user\_gate.py`, #141 ?), held-out-USER learning curve (`backend/eval/per\_user\_eval.py`, #142 ?), per-user no-regression guard (in flight). Heavyweight VLM personalization (on-device Qwen3-VL fine-tune) is **explicitly** out of v1 scope ? v1 personalization is the cheap LogReg/threshold/cue-ON-OFF layer only, i.e. *exactly* a thin overlay on Gen-0.

The coupling that makes M0.4 sequencing matter: a per-user `LogReg` is fit on the **Gen-0 feature schema**. The ActionFormer swap (M0.4) is a **feature-schema change** ? it bumps `FEATURE\_SCHEMA\_VERSION` ? it **invalidates every per-user LogReg** trained on v1 (the `feature\_schema\_hash` in the `user\_models` store is precisely the MINOR-carry vs MAJOR-refit switch). This is the #1 cross-track risk (\$7).

6. Milestones + the honest ship gate

Sequenced (each = ONE PR off `master`, owner approval first; no PR stacking):

Step	Work	Where it runs	Gate to advance
M0.1	Corpus spine + event-index contract (collector)	this container (pure-logic)	round-trip + split-by-match + no-Gemini-rows tests pass
M0.3a	Gemini-silver purge/retarget (\$2)	this container (pure-logic)	one clean training door + CI guard rejecting `source=gemini`
M0.3b	WASB C3 carried as ship blocker	owner workstream	tracked, gates <i>ship</i> not research
M0.4-1	Ratify the ship-gate F1@tIoU target (the open decision)	owner discussion	a <i>number</i> exists
M0.4-2	ActionFormer/ASFormer swap inside the existing harness	GPU/WSL box	re-runs green; LOVO recorded; schema-version bumped
Phase 1	Gen-0 ship decision A/B	GPU/WSL box	gate clears <i>with</i> significance

The honest ship gate (already enforced in `training/gen0/harness.py::gate\_verdict` ? keep it honest, don't soften it):

- **FLOOR** (necessary, not sufficient): LOVO mean F1 > heuristic floor **0.480**. Gen-0 passes (0.626).
- **PAIRED PER-VIDEO SIGN TEST** (the binding stat at small N): one-sided $p < 0.05$. **Currently** 5/6 wins, $p=0.109$? NOT significant. This is *why* corpus growth gates the ship, not method.
- **EXPLICIT F1@tIoU TARGET** (the open M0.4 decision): beating the floor is *necessary*, not sufficient. Suggested shape (owner to ratify/adjust, from `NEXT\_STEPS` #2): **F1@tIoU=0.5**

? 0.70 on multi-court folds, paired-sign $p < 0.05$ vs heuristic."** Reference points: floor 0.480 (necessary), **multi-court wrapper 0.66 (the number that makes the owned model worth serving)**,
 clean-footage 0.93 (Gemini's tier, NOT our target). Tighter tIoU for precise rally cuts than for highlights.

- **HARD ATTESTATION BLOCKS (cannot be out-scored):** WASB C3 unresolved ? commercial ship refused regardless of F1; consent ledger / minors-exclusion attested. The harness **never grades itself into serving** ? the product side (`backend/eval/regression.py`) owns final admission. Keep that separation.

7. RISKS (honest)

1. **Compute / budget to reach Gen-2 = the LARGEST open risk, and it's unquantified.** Phase-0 is \$0/local and dodges it entirely ? but the moment the VLM is on the table we need a **real VRAM/token + \$/call benchmark** on rented A100/L40S and the **self-hosted-VLM-vs-Gemini crossover**. ms-swift multimodal-GRPO uses 678 GPUs. *Do not stand up an owned serving stack until these clear.* (This is the strongest argument for staying on the head in Phase-0.)
2. **Small-N statistics.** $n=6$? the mean gap (0.626 vs 0.480) is inside noise (SE ~ 0.11); the sign test is the honest stat and it's **not yet significant** ($p=0.109$). Mitigation is *data*,
 not cleverness: $n=10/9$ -wins reaches $p \geq 0.011$. Per-user eval is the *same* small-N regime but **worse** (sharding when you should pool) ? `PERSONALIZATION\_PLAN` PR-0.5 "confirms-or-kills" the per-user premise before any serving build-out.
3. **The M0.4 ActionFormer feature-schema change invalidates per-user LogRegs.** A schema bump (`FEATURE\_SCHEMA\_VERSION` 1?2) breaks train==serve for every v1 per-user model; the `feature\_schema\_hash` MINOR/MAJOR switch in `user\_models` must drive a refit, and baseline-upgrade governance (MINOR carry / MAJOR re-fit) is still a Phase-3 personalization item. **Sequence the ActionFormer swap and per-user serving deliberately** so we don't ship a per-user layer that the next baseline upgrade silently corrupts.
4. **WASB C3 provenance** blocks *commercial ship* (not research); own-weights is an un-scoped workstream that **multiplies per sport** and (ADR-002) *can't* be trained from our temporal labels. Don't discover this late.
5. **Multisport temporal generalization is an OPEN HYPOTHESIS** (corpus is badminton-only; RacketVision was *refuted on verification* ? don't cite it as evidence). Sound on first principles; validate only once a 2nd-sport corpus exists. Per-sport **clean trajectory detectors** (tennis/TT/pickleball/padel) are *not yet identified* ? the binding multisport blocker.
6. **Base-model pretraining provenance is unauditable for every open VLM (incl. Qwen3-VL).** Our "no paid-LLM training" rule is enforceable at *our fine-tune layer*, not the base's pretraining ? an **explicit owner risk-acceptance** is required before building on any open VLM (a Gen-2 gate, not a Phase-0 one).

TOP 5 DISCUSSION POINTS FOR THE OWNER

1. **Greenlight which Phase-0 milestones, in what order?** Recommendation: **M0.1 (corpus + event-index contract) ? M0.3 (Gemini-silver purge) ? ratify the M0.4 ship-gate target ? M0.4 ActionFormer swap.** M0.2 is done. Each is one PR; none starts without your nod.
2. **Ratify the ship-gate F1@tIoU target (the standing open decision).** Beating the floor 0.480 is *necessary, not sufficient*. Proposed: ***LOVO F1@tIoU=0.5 ? 0.70 on multi-court folds, paired-sign $p < 0.05$ vs heuristic,** anchored to the multi-court wrapper **0.66** (the number that makes the owned model worth serving). Adjust the number?
3. **Approve the M0.3 purge shape.** Recommendation: route **all training** through the already-clean `training/gen0/harness.py`; demote `distill\_local`/`calibrate\_local` to **eval-only** references with corrected (de-Gemini-fied) docstrings; add a **CI guard that rejects any `source=gemini` row from the training view** (= the M0.1 acceptance test). Nothing Gemini-trained is in git today, but the path is one command away ? this closes it.
4. **Confirm corpus growth is the priority lever over method.** The math says so: $n=6 \geq 10$ flips

the sign test from $p=0.109$ to ≈ 0.011 . The ActionFormer swap is worth doing, but the *ship decision* is gated on **data**, not architecture. Agree to front-load Roora multi-court + TT growth?

5. **Acknowledge the schema-coupling sequencing risk.** The M0.4 ActionFormer swap **bumps the feature schema and invalidates every per-user LogReg** (per `PERSONALIZATION\_PLAN`). Decide the order of "ActionFormer baseline upgrade" vs "per-user serving build-out" *now* so we don't ship a per-user layer the next baseline silently breaks ? and confirm **compute/budget for Gen-2 stays out of Phase-0** (it's the largest unquantified risk; the head keeps us at \$0).

25. user (2026-06-14T10:44:43.072Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\OWNED_MODEL_PHASE0_PLAN.md (file state is current in your context ? no need to Read it back)

26. assistant (2026-06-14T10:45:03.575Z)

Plan written to `C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\OWNED\_MODEL\_PHASE0\_PLAN.md` (scratch only ? no code, no PRs). Grounded in a code-level read of the cited drivers, so the M0.3 section corrects a stale roadmap pointer.

Exec summary (~300 words)

Owned-Model Phase-0 is not "build the model" ? the Gen-0 head already exists and ships a versioned artifact (`artifacts/rally\_tal\_gen0/0.1.0/`, LOVO **0.626** vs heuristic floor **0.480**, `ship=False` by design). Phase-0 is "clear the pre-flight blockers and define the gate so the built head can be *trusted* and *grown*," staying \$0/local on the 2070 Super.

The **base choice** is settled and already wired: a **frozen WASB shuttle substrate (MIT)** + a single-sourced featurizer (`backend/features/rally\_seq.py`, train==serve, schema-versioned) + an **owned lightweight temporal head** (LogReg ? ActionFormer/ASFormer, all MIT). The **heavyweight Qwen3-VL/on-device VLM is deferred** because boundary accuracy *is* the product and fine-tuned VLMs miss strict boundaries ($R@0.7 \approx 50\%$ open-domain), the head is already de-risked (AUC 0.83 $\approx$ 0.92), and the VLM's compute/budget is the largest unquantified risk.

The **M0.3 Gemini-silver purge** is the hard pre-flight blocker. Code read confirms: nothing Gemini-trained is in git and the current Gen-0 harness already trains on **human golden labels** (`no\_paid\_llm\_training\_labels: true`). The residual risk is the **legacy drivers** `distill\_local.py` / `calibrate\_local.py` ? their docstrings, defaults, and `output/local\_rally\_model.json` output advertise the "distill Gemini ? student" path even though the code is label-source-agnostic. The roadmap's old "`training.label\_candidates` is the violation" pointer is **wrong** (it's a pure function) ? corrected in the plan. Fix: one clean training door (the harness), demote the drivers to eval-only with corrected docstrings, add a CI guard rejecting `source=gemini` training rows.

Data: owned $n=6$ (262 rallies) + Roora growth; MIT/Apache/BSD-only, minors excluded, split-by-match; the moat = consented dataset + harness, not weights. **Personalization** = a thin per-user `Variant` overlay on the Gen-0 baseline. **Ship gate**: floor 0.480 (necessary), paired sign test (currently 5/6, $p=0.109$? not significant), plus an explicit F1@tIoU target (the open decision).

Top 5 discussion points for the owner

1. **Greenlight which Phase-0 milestones, in what order?** Rec: M0.1 corpus+event-index ? M0.3 purge ? ratify M0.4 ship-gate target ? M0.4 ActionFormer swap. (M0.2 done; each is one PR.)
2. **Ratify the ship-gate F1@tIoU target.** Proposed: "LOVO F1@tIoU=0.5 ? 0.70 on multi-court folds, paired-sign $p<0.05$," anchored to the multi-court wrapper 0.66.
3. **Approve the M0.3 purge shape:** one clean training door (the harness), drivers ? eval-only with de-Gemini-fied docstrings, CI guard rejecting `source=gemini`.
4. **Confirm corpus growth > method as the priority lever** ? $n=6 \approx 10$ flips the sign test from $p=0.109$ to ≈ 0.011 ; the ship decision is gated on data, not architecture.
5. **Acknowledge the schema-coupling sequencing risk:** the ActionFormer swap bumps

`FEATURE\_SCHEMA\_VERSION` and invalidates every per-user LogReg ? sequence the baseline upgrade vs per-user serving now, and keep Gen-2 compute/budget out of Phase-0.

Key files cited (all absolute): `docs\OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md`, `docs\NEXT\_STEPS.md`,
`docs\PERSONALIZATION\_PLAN.md`, `backend\eval\distill\_local.py`,
`backend\eval\calibrate\_local.py`, `backend\eval\classifier.py`,
`backend\eval\fusion\_golden.py`, `training\gen0\harness.py`, `backend\features\rally\_seq.py`,
`eval\_baselines\heuristic\_lovo\_n6.json`, `artifacts\rally\_tal\_gen0\0.1.0\manifest.json`.